



INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo

ESCOM



Materia: Bases de Datos

Profesor: Gabriel Hurtado Aviles

Grupo: 3CV4

Título:

Práctica 1: "Modelo Entidad Relación"

Estado Del Arte

Miembros de Equipo:

Gonzalez Carapia Brandon Uziel

Introducción

A lo largo de este documento se revisará y analizará en profundidad el estado del arte en el ámbito de las bases de datos mediante el estudio riguroso de tres artículos científicos arbitrados publicados en congresos y revistas internacionales de alto impacto (ACM SIGMOD, IJACSA e IEEE Transactions on Knowledge and Data Engineering). Se aplicará lo investigado para conectar los avances tecnológicos recientes con los temas de la Unidad Temática I de nuestro programa de estudios, profundizando mediante casos de uso concretos y ejemplos prácticos en la resolución de problemas de autoconfiguración (auto-tuning) mediante aprendizaje automático a gran escala, la comparativa empírica entre paradigmas Relacionales y NoSQL, y la evolución hacia arquitecturas multimodelo.

Práctica 1: "Modelo Entidad Relación"

Objetivo: Que usted describa los sistemas de base de datos a partir de sus aspectos básicos, construya el entorno de trabajo que empleará durante todo el semestre —control de versiones y un sistema gestor de bases de datos ejecutado en un contenedor—, sustente su investigación con literatura científica arbitrada y aplique lo anterior al modelado conceptual de un problema real. Toda la evidencia se entrega en un repositorio de Git.

Esta práctica cubre la Unidad Temática I completa y, al mismo tiempo, establece la forma de trabajo del resto del curso. Se divide en cinco ejercicios: los dos primeros construyen el entorno técnico, el tercero y el cuarto sustentan la parte conceptual, y el quinto la aplica a un caso concreto. La entrega no es un archivo suelto: es un repositorio de Git donde quedará registrado cómo trabajó usted, no solo el resultado final.

Ejercicio 4. ESTADO DEL ARTE

Fichas Bibliográficas de los Artículos Seleccionados

Ficha Bibliográfica 1: Autoconfiguración de SGBD mediante Aprendizaje Automático a Gran Escala

1. Cita APA 7

Van Aken, D., Pavlo, A., Gordon, G. J., & Zhang, B. (2017). Automatic database management system tuning through large-scale machine learning. In Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD '17), 1009–1024. <https://doi.org/10.1145/3035918.3064029>

2. Problema que aborda

Los SGBD modernos cuentan con cientos de parámetros de configuración ajustables (como `shared_buffers`, `work_mem`, `max_worker_threads` en PostgreSQL o `innodb_buffer_pool_size` en MySQL). La interacción no lineal entre estas perillas sobrepasa la capacidad humana de los administradores (DBA) para encontrar la combinación óptima ante cargas cambiantes.

Ejemplo práctico: Si una base de datos pasa de procesar compras bancarias en tiempo real (cargas OLTP con escrituras rápidas y pequeñas) a generar reportes analíticos masivos de fin de año (cargas OLAP con lecturas pesadas y uniones de tablas), los parámetros de memoria asignados manualmente por el DBA se vuelven ineficientes. Ajustar la memoria de búfer (`shared_buffers`) sin modificar la memoria de ordenamiento (`work_mem`) puede agotar la RAM física o provocar escrituras excesivas en disco temporal, degradando severamente el tiempo de respuesta del sistema.

3. Método o propuesta

Proponen un sistema (OtterTune) que automatiza el proceso de tuneo recolectando métricas del SGBD en tiempo real (como páginas leídas/escritas, bloqueos y uso de CPU). Utiliza Factorización de Matrices para identificar las 'perillas' verdaderamente influyentes y Procesos Gaussianos para predecir la configuración óptima sin reiniciar el motor a ciegas.

Ejemplo práctico: En lugar de probar a ciegas miles de combinaciones, OtterTune compara la carga actual de un servidor e-commerce durante el 'Buen Fin' con historiales de otros servidores. Identifica que solo 10 de las 500 perillas afectan el rendimiento actual (por ejemplo, ajustando la memoria de ordenamiento y el límite de conexiones) y aplica automáticamente la mejor configuración aprendida sin interrumpir el servicio.

4. Resultado principal	Demuestran que OtterTune genera configuraciones que igualan o superan el rendimiento alcanzado por administradores de BD humanos experimentados, reduciendo la latencia de las consultas en hasta un 40% tanto en cargas OLTP como OLAP.
5. Relación con Unidad I	Se vincula directamente con el Tema de Sistema Gestor de Bases de Datos, específicamente en las funciones del Administrador de la BD (DBA) y los módulos del Gestor de Almacenamiento y Optimizador de Consultas.
6. Aporte al proyecto	Muestra la importancia de configurar adecuadamente los parámetros de memoria e hilos del motor (PostgreSQL) para evitar cuellos de botella durante la ejecución del caso de estudio del proyecto.

Ficha Bibliográfica 2: Evaluación Comparativa Empírica entre Bases de Datos Relacionales y NoSQL

1. Cita APA 7	Mihai, G. (2020). Comparison between relational and NoSQL databases. Annals of "Dunarea de Jos" University of Galati. Fascicle I. Economics and Applied Informatics, 26(3), 38–42. https://doi.org/10.35219/eai15840409134
2. Problema que aborda	Las bases de datos relacionales alcanzaron sus límites ante el volumen, velocidad y variedad masiva de datos no estructurados en aplicaciones web modernas, por lo que las organizaciones requieren criterios de selección claros entre el modelo relacional tradicional y los diferentes modelos NoSQL (clave-valor, documentos, grafos y columnas). Ejemplo práctico: Una empresa de logística no sabe si almacenar su catálogo de productos y el rastreo de paquetes GPS en un motor relacional como PostgreSQL o migrar a motores NoSQL como MongoDB (documentos) o Cassandra (columnas). La falta de análisis sobre disponibilidad, escalabilidad y consistencia lleva a decisiones basadas en tendencias tecnológicas en lugar de necesidades operativas reales.
3. Método o propuesta	Realiza un análisis comparativo estructurado evaluando las características operativas de las bases SQL y NoSQL según el teorema CAP y el modelo BASE, sintetizando además una taxonomía entre el modelo relacional y las 4 familias principales de almacenamiento NoSQL (Key-Value, Document, Graph y Wide-Column). Ejemplo práctico: Contrasta cómo una base relacional ejecuta operaciones transaccionales mediante SQL y uniones complejas (JOINS) garantizando ACID, frente a cómo un almacén de documentos

	tipo JSON gestiona catálogos flexibles permitiendo agregar atributos en tiempo de ejecución sin alterar el esquema ni interrumpir la aplicación.
4. Resultado principal	Demuestra que las bases NoSQL son superiores para aplicaciones web distribuidas a gran escala con esquemas flexibles y alta demanda de escalamiento horizontal y disponibilidad permanente (modelo BASE), mientras que las bases de datos relacionales siguen siendo la mejor opción para sistemas OLTP centralizados con datos estructurados y operaciones transaccionales estrictas (propiedades ACID).
5. Relación con Unidad I	Se relaciona de forma explícita con el Tema de Archivos frente a bases de datos y el Tema de Modelos de datos: su importancia y su clasificación por estructura (Relacional vs. NoSQL).
6. Aporte al proyecto	Justifica formalmente la selección de un SGBD Relacional (PostgreSQL) para nuestro caso de estudio, garantizando la integridad de las transacciones sin perder rendimiento en las operaciones del sistema.

Ficha Bibliográfica 3: Estado del Arte de los Sistemas de Bases de Datos Multimodelo

1. Cita APA 7	Mihai, G. (2020). Multi-model database systems: The state of affairs. Annals of "Dunarea de Jos" University of Galati. Fascicle I. Economics and Applied Informatics, 26(2), 211–215. https://doi.org/10.35219/eai15840409128
2. Problema que aborda	El problema de la 'persistencia políglota' (Polyglot Persistence), donde el uso de múltiples motores de bases de datos independientes (relacionales y NoSQL) dentro de una misma organización genera infraestructuras técnicas complejas, costosas y difíciles de mantener. Ejemplo práctico: Una empresa médica guarda datos transaccionales en PostgreSQL, documentos clínicos en MongoDB y redes de contactos en Neo4j. Sincronizar estos tres motores requiere un esfuerzo de programación excesivo a nivel de aplicación, incrementando los costos operativos y el riesgo de fallas de consistencia.

3. Método o propuesta

Realiza un análisis cuantitativo del mercado evaluando tres tendencias de convergencia: la adaptación de SGBD relacionales tradicionales para soportar modelos NoSQL (como JSON o grafos), la extensión de motores NoSQL nativos hacia otros modelos, y el desarrollo de nuevos motores multimodelo nativos.

Ejemplo práctico: Analiza cómo SGBD relacionales líderes (como Oracle o PostgreSQL) han integrado tipos de datos JSONB o motores de grafos, permitiendo consultar datos relacionales y documentos no estructurados mediante una única plataforma y lenguaje SQL unificado.

4. Resultado principal

Muestra que, aunque la convergencia completa hacia sistemas multimodelo unificados toma tiempo, los SGBD relacionales consolidados han sido los más eficaces en adoptar modelos NoSQL (19.5% de los sistemas RDBMS integran otros modelos), permitiendo a las organizaciones reducir la complejidad de la persistencia políglota.

5. Relación con Unidad I

Se vincula directamente con el Tema de Arquitectura de tres niveles e independencia de datos y con el Tema de Clasificación de los modelos de datos.

6. Aporte al proyecto

Ofrece el respaldo técnico para aprovechar las capacidades multimodelo de PostgreSQL (por ejemplo, columnas de tipo JSONB) dentro del diseño lógico-físico de la base de datos del proyecto.

Análisis Comparativo del Estado del Arte y Problema Abierto

- Puntos en Común: Los tres artículos investigados se enfocan en superar las limitaciones operativas de los sistemas tradicionales de bases de datos ante el crecimiento exponencial en volumen, velocidad y variedad de datos. Coinciden en la necesidad de evolucionar la gestión (ya sea mediante inteligencia artificial para autoconfiguración de parámetros, análisis y evaluación de paradigmas de almacenamiento NoSQL vs. SQL, o la adopción de arquitecturas multimodelo unificadas) para maximizar la eficiencia sin comprometer la integridad.
- Diferencias: Van Aken et al. (2017) abordan el problema a nivel de infraestructura y administración interna, optimizando los parámetros del motor mediante aprendizaje automático. Mihai (2020, Fascículo 3) analiza el dilema a nivel macro-arquitectónico comparando el modelo relacional frente a las distintas familias NoSQL en términos de escalabilidad y transaccionalidad. Por su parte, Mihai (2020, Fascículo 2) analiza cuantitativamente la convergencia del mercado de software hacia sistemas gestores multimodelo capaces de soportar varios paradigmas en un solo motor.
- Problema Abierto Identificado: A partir del estudio comparativo de estos trabajos, se identifica como un problema abierto la Autoconfiguración Automatizada por IA en Bases de Datos Multimodelo Distribuidas. Específicamente, cómo lograr que algoritmos de machine

learning (como OtterTune) ajusten dinámicamente en tiempo real los parámetros del motor cuando este maneja de forma simultánea cargas de trabajo transaccionales SQL (relacionales) y consultas documentales NoSQL (JSON) en entornos distribuidos masivos, sin degradar las garantías de consistencia ACID ni la alta disponibilidad.

Conclusión

Gonzalez Carapia Brandon Uziel:

Durante el desarrollo de este ejercicio logré analizar y documentar el estado del arte en el área de las bases de datos mediante la revisión crítica de tres artículos científicos arbitrados (ACM SIGMOD y Annals of "Dunarea de Jos" University of Galati). Comprendí cómo la investigación actual está transformando la gestión de SGBD a través de la autoconfiguración automatizada por IA y con ayuda de esta como el análisis riguroso entre enfoques SQL/NoSQL va mejorando cada vez mas ademas de que la convergencia hacia arquitecturas se va polarizando hacia un sistema de multimodelo. Asimismo, la vinculación explícita de cada estudio con los temas de la Unidad Temática I reforzó mi capacidad para conectar la teoría revisada en clase con la frontera del conocimiento tecnológico actual.